LiteScript cheatsheet

JavaScript-like scripting language embedded in C++ applications. The language has **no built-in functions**: the application provides them (and may enable the standard library). Full docs: docs/README.md.

Syntax essentials

Statements	`;` is **mandatory**: `var a = 1;`
Comments	`// line`, `/* block */`
Variables	`var a;` (null) · `var b = 2, c = "x";` — no `let` / `const`
Names	`[A-Za-z_$][A-Za-z0-9_$]*`
Scope	`var` is local to the function (or the current namespace outside functions)
Lookup	function → current namespace → global namespace
Blocks	braces optional for a single statement
Script result	`return value;` at the top level stops the script and gives the value to the application

Keywords: `var function class static operator new this return if else for while do switch case default break continue try catch throw namespace true false null undefined`

Not in LiteScript: semicolon-less code, prototypes, arrow functions `=>`, `let` / `const`, `finally`, string → number implicit conversion, short-circuit `&&` / `||`.

Types

Type (`typeOf`)	Literals	Copy
`undefined`	`undefined` — missing variable / member / argument	value
`null`	`null` — `var x;`, impossible operation	value
`boolean`	`true`, `false`	value
`number`	`42`, `-6`, `2.5`, `.8e-3` — int64 or double	value
`string`	`"Hello"`, `'¡Hola!'` — UTF-8	value
`callback`	`function(x) { return x; }`	—
`array`	`[1, 2]`, `{ name: "Percy", "age": 17 }`	**shared**
`class`	`class { ... }`	**shared**
`class object`	`new Point(1, 2)`	**shared**
`namespace`	`namespace Math;`	**shared**

String escapes: `\n \t \r \b \f \v \0 \uXXXX \u{1F600}`, any other `\c` gives `c` (`\\`, `\"`, `\'`).

Falsy: `undefined`, `null`, `false`, `0`, `NaN`, `""`. Everything else is true (empty arrays too).

Arrays & strings

```
var a = [10, 20, 30];
var o = { name: "Percy", "age": 17 };
a[0]; a.length;          // 10, 3 (indexed elements only)
o.name; o["age"];        // "Percy", 17
a[5] = 60;               // a[3], a[4] undefined, len 6
o.city = "NY";           // adds a named element
a[10];                   // undefined, NOT created
[1, 2] + { x: 3 };       // new array with both contents

var b = a; b.push(4);    // a changes too (shared)

var s = "héllo";
s.length; s[1];          // 5, "é" (characters)
s[0] = "H";              // strings are mutable values
"ab" * 3;                // "ababab"
```

Operators

Lower precedence first. Same level: left → right, except `?:` and assignments (right → left).

Prec.	Operators → overload name
1	`a++` post_increment · `a--` post_decrement · `a(...)` call · `a[b]` get · `a.b`
2	`++a` pre_increment · `--a` pre_decrement · `+a` unary_plus · `-a` unary_minus · `!a` not · `~a` bitwise_not · `new A()`
3	`*` multiply · `/` divide · `%` modulo
4	`+` add · `-` subtract
5	`<<` left_shift · `>>` right_shift
6	`>` greater · `<` lesser · `>=` greater_equal · `<=` lesser_equal
7	`==` equal · `!=` not_equal
8–10	`&` bitwise_and · `^` bitwise_xor · `|` bitwise_or
11–12	`&&` and · `||` or
13	`a ? b : c`
14	`=` assign · `+=` add_assign · `-=` subtract_assign · `*=` multiply_assign · `/=` divide_assign

Behavior	Example
int ÷ int stays int if exact	`6 / 2` → `3`, `7 / 2` → `3.5`
int64 overflow → float, ÷0 → `Infinity` / `NaN`	`1 / 0`
Incompatible types → `null` (no error)	`1 + "2"` → `null`, `-"a"` → `null`
String + anything → concatenation	`"a" + 1` → `"a1"`
`==` never converts	`1 == "1"` false, `1 == 1.0` true
Arrays / instances / classes compare by identity	`[1] == [1]` false
`< > <= >=` only number/number or string/string	otherwise false
`! && ||` give a boolean, **both sides evaluated**	
Default value: `|` with non-number left side	`opts.name | "default"`

Control structures

```
if (c) { ... } else if (d) { ... } else { ... }
while (c) { ... }
do { ... } while (c);
for (var i = 0; i < 10; i++) { ... }
for (;;) { ... }             // no condition = true
switch (v) {                 // cases: any expr, ==
    case 1:
    case f(2): ...; break;   // no break: fall through
    default: ...             // must be last
}
break; continue;             // continue ok in switch/try
```

Functions

```
function add(a, b) { return a + b; }
var sub = function(a, b) { return a - b; };
var eight = (function(x) { return x * 2; })(4);

f();                         // missing args are undefined
f(1, 2, 3, 4);               // extra args are ignored
function g(a, rest...) { }   // g(1, 2, 3): rest = [2, 3]

function counter() {     // closures
    var n = 0;
    return function() { n++; return n; };
}

var arr = [3, 1] + {
    first: function() { return this[0]; }
};
arr.first();                 // 3: this = arr
```

- `this` = the array/instance the function was read from, else `undefined`.
- No `return` → `undefined`.
- Arguments are copies, but arrays/instances they hold are shared.
- Max call depth set by the application (100,000 by default).

Classes

```
class Point : Base, Other {   // multiple inheritance
    var x = 0, y = 0;         // copied per instance
    static var count = 0;

    function Point(x, y) {    // constructor
        this.Base();          // inherited constructor
        this.x = x; this.y = y;
        count++;              // "Point." optional
    }
    function len() { return this.x + this.y; }
    static function zero() { return new Point(0, 0); }

    operator add(p) {         // p1 + p2
        return new Point(this.x + p.x, this.y + p.y);
    }
    operator equal(p) { return this.x == p.x; }
    operator get(i) { return i == 0 ? this.x : this.y; }
    operator call(k) { return this.x * k; }  // p(2)

    class Inner { }           // nested = static
}

var p = new Point(1, 2) + Point.zero();
p.label = "A";                // add a member

var Anon = class {
    function constructor(v) { this.v = v; }
};
var once = new (class { var a = 1; })();
```

- Constructor = method named like the class (`constructor` for an anonymous class).
- Bases are any expressions; all members are public; inherited methods/operators can be redefined.
- Overloaded operator only used if the instance is the **first** operand (`5 + p` → `null`).
- Without overload: `==` / `!=` compare identity, `=` replaces the value.

Namespaces

```
namespace Math;           // enter (created if needed)
function add(a, b) { return a + b; }
namespace Game.Engine;    // nested
var version = 2;
namespace global;         // back to the default

Math.add(3, 2);           // 5
Game.Engine.version;      // 2
Math.zero = 0;            // add from outside
Math = 5;                 // no effect
```

Errors

```
throw "invalid value";        // any value
throw { message: "bad", code: 3 };
try { risky(); }
catch (e) { print(e.message, e.line, e.column); }
try { risky(); } catch { }    // name optional
```

- No `finally` . Errors propagate up calls to the nearest `try` ; `catch` may rethrow.
- Engine errors are arrays `{ message, line, column }` (1-based position).
- Uncaught errors stop the script and go to the application.

Engine error	Cause
`a value of type number isn't a function`	`x()` on a non-function
`can't read the member "y" of undefined`	`x.y` on `undefined` / `null`
`can't read an element of null`	`x[0]` on `undefined` / `null`
`a value of type array isn't a class`	`new x()` on a non-class
`the instance doesn't overload the call operator`	calling an instance
`maximum call depth exceeded`	too much recursion

Standard library (optional)

Math

`abs sign floor ceil round trunc`	`Math.floor(2.7)` → `2`
`sqrt pow exp log`	`Math.pow(2, 10)` → `1024`

Math

`sin cos tan asin acos atan atan2(y, x)`	radians
`min(a, b, ...)` , `max(a, b, ...)`	
`random()` / `random(n)` / `random(a, b)`	[0, 1[· int [0, n[· int [a, b]
`seed(n)` , `PI` , `E`	non-number argument → `null`

Conversions

`typeOf(x)`	type name (see Types)
`toString(x)`	text of a value
`parseNumber(s)`	`" 4.5 "` → `4.5` , else `null`
`parseInt(s, base)`	base 2–36 (default 10)
`isNaN(x)`	not a number or `NaN`
`char(code)`	`char(233)` → `"é"`

Array methods

`push(x, ...)` / `pop()`	append (→ new size) / remove last
`insert(i, x)` / `remove(i)`	`remove(-1)` = last
`clear()`	remove indexed elements
`indexOf(x)` / `contains(x)`	`-1` if absent
`join(sep = ",")`	text of the elements
`slice(start, end)`	negative = from the end
`reverse()` / `sort(cmp)`	in place, stable; `cmp(a, b) < 0` → a first
`keys()`	names of the named elements
`map(f)` / `filter(f)` / `forEach(f)`	`f(element, index)`

A named element hides a method with the same name.

String methods (return new strings)

`substring(start, end)`	part of the string
`indexOf(t, from)`	`-1` if absent
`contains(t)` , `startsWith(t)` , `endsWith(t)`	search
`split(sep)`	`""` → one element per character
`replace(a, b)`	replaces **all** occurrences
`trim()` , `toUpperCase()` , `toLowerCase()`	
`charCode(i)`	Unicode code at `i`

Playground (`litescript-pg`)

Runs `.lsc` files with the standard library **plus** `print`, `assert`, `Console`, `File`, `Directory`, `Path`, `JSON`, `Http`, `Process`, `Time`.

Command line

```
litescript-pg script.lsc              # run a file
litescript-pg "dir/*.lsc" "t/**/*.lsc" # patterns
litescript-pg dir                     # every .lsc of dir
litescript-pg -e "print(1 + 2);"      # run code
litescript-pg a.lsc -- one two        # Process.args
litescript-pg                         # interactive
```

Option	
`-e`, `--eval <code>`	run code before the files (repeatable)
`-s`, `--shared`	files share one state (isolated by default)
`-b`, `--bail`	stop at the first failing file
`-i`, `--interactive`	interactive session after the files
`-h` / `-v`	help / version
`--lsp` / `--dap`	language server / debugger (editors)
`--disassemble f`	instructions of a script

- Files run in alphabetical order. A first `#!` line is ignored.
- Errors: `file:line:column: error: message`. Exit code 1 if a file failed, 2 for a bad command line.
- Interactive: an empty line runs what was typed; a single expression's value is printed and stored in `_`.
- Build: `scripts/build.sh` (`.bat` on Windows) → `builddir/release/playground/litescript-pg`; `-Dplayground=false`, `-Dhttp=disabled`.

Console

`print(a, b, ...)`	values separated by spaces + newline
`assert(cond, msg)`	throws `assertion failed: msg`
`Console.write(...)` / `writeLine(...)`	without / with newline
`Console.error(...)`	to stderr
`Console.readLine(prompt)`	one line, `null` at end of input
`Console.readAll()`	whole stdin

Files & paths

Paths relative to the current directory, `/` as separator.

File	
`read(p)` / `readLines(p)`	text / array of lines
`write(p, text)` / `append(p, text)`	replace / add at end
`exists(p)` / `size(p)`	bytes
`copy(src, dst, overwrite = true)` / `move(src, dst)`	
`remove(p)`	`false` if missing

Directory	
`list(p = ".", recursive)`	sorted paths
`create(p)`	with parents, `false` if exists
`remove(p, recursive)`	`false` if missing
`exists(p)`, `current()`, `setCurrent(p)`	

Path (`"a/b/c.lsc"`)	
`join(a, b, ...)`	
`directory` / `filename` / `stem` / `extension`	`"a/b"` / `"c.lsc"` / `"c"` / `".lsc"`
`absolute(p)`, `normalize(p)`, `exists(p)`	`normalize` removes `.` / `..`

JSON

`JSON.parse(text)`	objects → named elements, arrays → indexed; invalid text throws (with position)
`JSON.stringify(v, indent)`	`indent`: spaces count or text (compact by default)

- Arrays with names → objects (indexes become names); instances/namespaces → objects of their members.
- Functions, classes, `undefined` skipped; self-containing value throws; `{}` is written `[]`.

HTTP

`Http.get(url, headers)` / `Http.delete(url, headers)`	no body
`Http.post(url, body, headers)` / `put` / `patch`	non-string body → JSON
`Http.request({ url, method, headers, body, timeout, followRedirects })`	
`Http.available`	`false` if built without HTTP

- Result: `{ status, ok, headers, body }` — `ok` for 2xx, header names lower case.
- Timeout in ms (30 s default), redirects followed. Network error throws.

Process & time

`Process.args`	arguments after `--`
`Process.script`	absolute path of the running script
`Process.platform`	`"windows"`, `"linux"`, `"macos"`, `"unknown"`
`Process.env(name)`	`null` if undefined
`Process.run(cmd)`	`{ code, output }` (stdout)
`Process.exit(code)`	stop the program
`Time.now()`	ms since epoch (UTC)
`Time.clock()`	monotonic ms, for durations
`Time.sleep(ms)`	wait
`Time.iso(ms)`	`"2024-05-17T08:30:00.000Z"` (now by default)

Recipes

```
// Arguments with default
var args = Process.args;
var name = args.length > 0 ? args[0] : "World";

// Read stdin until an empty line
while (true) {
    var line = Console.readLine("> ");
    if (line == null) break;    // no short-circuit
    if (line.trim() == "") break;
    var n = parseNumber(line);
    print(n == null ? "not a number" : n * 2);
}

// Edit a JSON file
var cfg = JSON.parse(File.read("config.json"));
cfg.version += 1;
File.write("config.json", JSON.stringify(cfg, 2));

// Walk a directory
Directory.list(".", true).forEach(function(p) {
    if (File.exists(p)) print(File.size(p), p);
});

// HTTP + JSON
var url = "https://api.github.com/repos/LePtitDev/lite-script";
var r = Http.get(url, { Accept: "application/json" });
if (r.ok) print(JSON.parse(r.body).stargazers_count);

// Time a command
var t = Time.clock();
var res = Process.run("git log --oneline -5");
if (res.code == 0) print(res.output);
print("Done in", Math.round(Time.clock() - t), "ms");
```